Jan-Øivind Lima

# Procedural generation: music

Generating music using method X and Y

Master's thesis in Cybernetics and Robotics
Supervisor: Sverre Hendseth
June 2026

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

NTNU

# ABSTRACT

Write an abstract/summary of your thesis, and state your main findings here.

A summary should be included in both English and any second language, if this is applicable, regardless if the thesis is written in English or in your preferred language. These should be on separate pages, the English version first.

# PREFACE

Write the preface of your thesis here.

You may include acknowledgements and thanks as part of your preface on this page, or you may add it as a new chapter after the preface.

# CONTENTS

**Abstract** **i**

**Preface** **ii**

**Contents** **iv**

**List of Figures** **iv**

**List of Tables** **v**

**Abbreviations** **vii**

**1 Introduction** **1**

**2 Theory** **3**
2.1 Music theory . . . . . 3
2.1.1 Notes . . . . . 3
2.1.2 Scales . . . . . 3
2.1.3 Keys . . . . . 4
2.1.4 Melodies . . . . . 4
2.1.5 Harmony . . . . . 4
2.2 Procedural generation . . . . . 5
2.2.1 Wave function collapse . . . . . 5
2.3 Object oriented programming . . . . . 5

**3 Software design** **7**
3.1 Object orientation . . . . . 7

**4 Implementation** **9**
4.1 The iteration process . . . . . 9

**5 Results** **11**
5.1 First iteration . . . . . 11
5.1.1 Features . . . . . 11
5.1.2 Limitations . . . . . 11
5.1.3 Specifications for the second iteration . . . . . 12

**6 Discussion** **13**
6.1 Future work . . . . . 13
6.1.1 Non-block harmony . . . . . 13

**7 Conclusions** **15**

**References** **17**

**Appendices:** **19**

**A - Github repository** **20**

**B - Audio examples** **21**

**C - Code documentation** **22**

# LIST OF FIGURES

2.1.1 The c-major scale written in the treble clef. . . . . 4
2.1.2 The eb-major scale written in the treble clef. . . . . 4
2.1.3 Harmonic progression in tonal music. The arrows indicate the typical flow of harmonic functions. . . . . 5

5.1.1 Example melody of the first iteration melody generator. . . . . 11

# LIST OF TABLES

# ABBREVIATIONS

List of all abbreviations in alphabetic order:

- **AI** Artificial Intelligence
- **LLM** Large Language Model
- **NTNU** Norwegian University of Science and Technology

CHAPTER

# ONE

# INTRODUCTION

For hundreds of years composers have been creating music using external tools like dice to help them generate something new. As early as the 18th century the idea of using randomness to help the composing process has been used. The musical dice game, "Musikalisches Würfelspiel", was used to randomly generate music based on precomposed parts. [1] This is a very basic way of generating music, however by the random nature of the dice it will not be able to create anything coherent and meaningful like a human composer can. Additionally this process only used precomposed parts which were designed to work together. If we want to generate everything from scratch we need a way more sophisticated system. This is where procedural generation comes into the picute. If we base the meldoy on something random and then use the conventions and "rules" for melodic composition we might be able to generate music that sounds somewhat coherent.

Today the most common way to go about generating music is using AI and LLM's. This enables anyone to create music, however these models are trained on large data sets and imitates what has come before. For this thesis the focus is on the craft of creating music, breaking it down to its very primitive form, only pitches and durations. This will enable the user to have more control over exactly what is being generated. However this requires us to create a system and models that capture the essence of how exactly we are to go about contructing the very basic musical ideas. It will not depend on large data sets and will create a musical ideas that can be processed into a fully fledged musical idea.

If we are able to solve this problem of creating a good structure and model for this kind of generation we will not only have a solution for musical ideas, but this kind of system can solve many kinds of problems.

The problem we want to solve is the problem of creating a coherent system that will be able to generate music that adheres to certain musical principles. In order to do this we need to consider many things, and system design is a big part of this. When buidling such a program we need to consider what parts it should consist of as well as how they are supposed to interact. Object oriented programming (OOP) is a solid way to approach this system design.

The solution will be a system that is modular, easy to modify and generate

new music using newly defined constraints or inputs on structure, motifs etc...

CHAPTER

# TWO

# THEORY

In order to understand how melodies are structured we need to lay some theoretical groundwork using music theory. Additionally we will cover some concepts regarding procedural generation, *this can include wave function collapse, or other things researched.*

## 2.1 Music theory

### 2.1.1 Notes

The very core of music is composed of notes. Notes are vibration in air which we interpret as pitches. In the western system we agree upon the standard of pitches is based around A4 which is $440Hz$. All other pitches are tuned in relation to this reference.

### 2.1.2 Scales

The musical scale is building block for creating not only melodies but also harmony, which we will cover a bit later. Firstly the musical scale is, for the most part, constructed using seven notes. In the western canon this is usually seven notes. You can see the basic c-major scale in Figure 2.1.1, it is comprised of the notes c d e f g a b c. The tones of the scale can be named according to their function within the scale. We often label the notes $\hat{1}, \hat{2}, \hat{3}, \hat{4}, \hat{5}, \hat{6}, \hat{7}$, and then starting back at $\hat{1}$. The use of the carets are called the Nashville Number System, this is used to differentiate the scale degrees from the chord tones which will be discussed in Section 2.1.5 When we consider chord structures we use the numbers but usually count above from 8 to signify harmonic functions, but when considering only scales and melodies we usually only use 1 through 7

The most important function is that of the scale degree 1, this is the *tonic*, which is the 'home' of the key. This is the most stable function there is, it feels resolved and does not have any tension urging it to move anywhere.

Next up we have the dominant function, there are two notes degrees which fall into this category, that is the 5th and 7th scale degree. These notes have tension. They want to move, especially the 7th scale degree, it has a great pull toward the

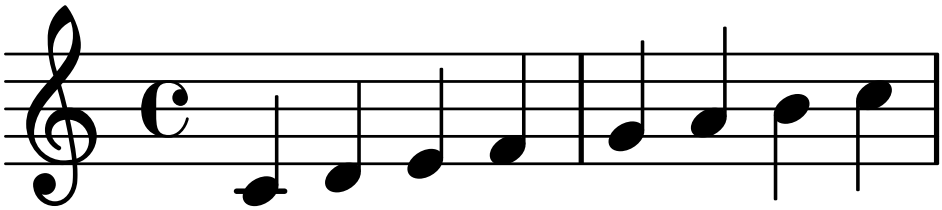

**Figure 2.1.1:** The c-major scale written in the treble clef.

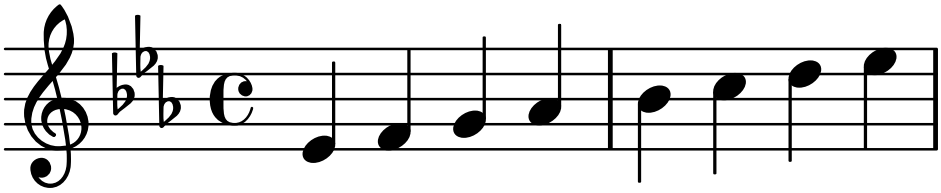

**Figure 2.1.2:** The eb-major scale written in the treble clef.

tonic. You can hear this in the example under Figure 2.1.1. The penultimate tone is the 7th scale degree and it leads to the tonic.

To lead us to the tonic functions we have what are called the pre-dominant function. This simply means 'before dominant', and their function is to set up the tension to lead us to the dominant degrees which in turn leads us back to the tonic.

### 2.1.3 Keys

The concept of a musical key is simply shifting around where the tonic note is. For instance in Figure 2.1.1 you can see the major scale in c. And in Figure 2.1.2 you can see the major scale in eb. In todays tuning system there are no differences between the keys, meaning that the sound the same, just higher and lower. This was not the case earlier in history, where the temperament(how the tones were tunes in relation to each other) of the keys differed. Today it just sounds higher or lower depending on the key.

### 2.1.4 Melodies

Melodies is the horizontal component of music, meaning we look at it as a time-series. We have a given note at a given length followed by another note or rest of a given length etc... This means that you can look at a melody as a series of intervals as well. If we do this on known melodies we will observe that they are mainly composed of steps and skips. This means that the distance between one note and the next is one or two tones. Large leaps are usually leaping to important tones like the tonic or the root of the current chord.

### 2.1.5 Harmony

Harmony is the vertical component of the music, harmony is what happens when two or more notes happen at the same time. This is what chords are build around.

When considering the harmonic progression in tonal music we can consider all the standard chords in the major key. They are labeled by numbers using the roman numerals. Upper case means the chord in major and lower case means minor. You can see the difference between major and minor chords in figure [make a figure for this], and listen to the difference at the link provided in the caption.

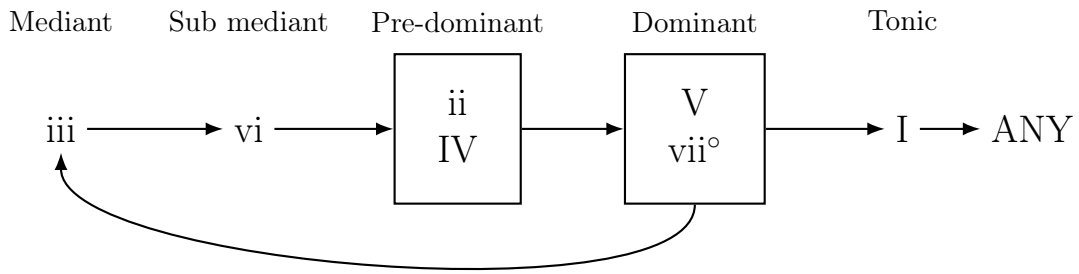


**Figure 2.1.3:** Harmonic progression in tonal music. The arrows indicate the typical flow of harmonic functions.

## 2.2 Procedural generation

This section covers the relevant theory regarding procedural generation.

### 2.2.1 Wave function collapse

Might use this in the proc-gen so a chapter on this might be useful.

## 2.3 Object oriented programming

CHAPTER

# THREE

# SOFTWARE DESIGN

Include the complete description of the methods used in your research here.

## 3.1 Object orientation

This project will use the widely known process of object oriented programming. In simple terms this simply means that all the parts of the code are structured as objects, meaning everything is represented as objects with given properties. This means that for instance a 'melody object' could consist of all the notes and durations it consists of, the key it is written in and any metadata explaining what the melody represents. Meaning if it is a verse or chorus etc...

In order to design an object oriented program the design process is very important. Creating good models for the objects are imperative to make it work. If the level of abstraction is too high, the sematic gap between the real-world concepts they represent and the acutal implementation in the code will be too large and the model will be un-intuitive and hard to both use and also understand.

CHAPTER

# FOUR

# IMPLEMENTATION

The implementation of this program started by just writing something that generated something. This was done to not get stuck in the planning phase. You could consider this a contradiction to the part in software design which states that good planning is imperative. However, trying out some code and figuring out what works, what doesn't and what needs to be implemented in the models is very helpful. Therefore the first iteration was a test to get some systems working and finding out what it neccesary to make it work and what is not imporatant.

## 4.1 The iteration process

After the first iteration lots of issues were observed and were put into the spec for the second iteration which improved upon the model in various ways. Some imporatnt features were restricting the range so that an acutal instrument can play it, be that an instrument or the human voice.

CHAPTER

# FIVE

# RESULTS

## 5.1 First iteration

After a few weeks of development the project grinded to a halt due to the framework established was incompatible with the ideas of how to improve the system. This led to the first refactor of the project. The code was working, but unable to be improved upon. I took what I learned from this first work and started planning the next iteration using the limitations of the first iteration as a spec for the next one.

The first iteration had some good features, but also lacked several important ones I found to be important after running into the first wall of the project.

### 5.1.1 Features

The system was able to generate random melodies using stepwise motion, meaning moving by 2nds, and skips, meaning moving by 3rds. This made for somewhat singable melodies.

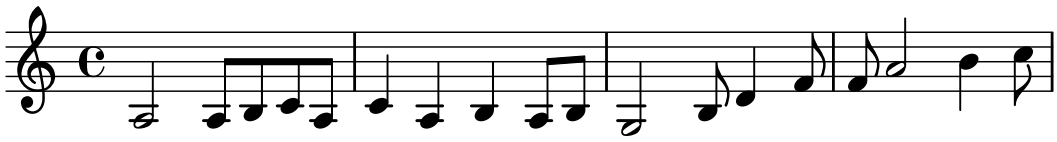

**Figure 5.1.1:** Example melody of the first iteration melody generator.

### 5.1.2 Limitations

The system had no way to track which octave it was in which meant that longer melodies had a tendency to meader outside what would be considered as singable or even playable register.

There were no consideration of tension and release in the melody. Phrases never

Rests were not implemented in any way, pauses and rests are an important part of a melody and this needs to be addressed so that the melodies are not simply a continous stream of notes.

### 5.1.3 Specifications for the second iteration

For the second iteration I wanted to focus on these things.

- No absolute notes until rendering(keep it at scale degrees, to reduce overhead)
- Octave tracking
- Restricted range
- Ending prases resolved(tonic, long note)
- Self similarity

CHAPTER

# SIX

# DISCUSSION

Discuss your results here.

The generated music is good because this:

The generated music is bad because this:

etc etc...

## 6.1 Future work

The current implementation is not complete and there are several features which would benefit the system.

### 6.1.1 Non-block harmony

*Not sure how far the project will go, but probably no implementatino of Non-block harmony happened.*

CHAPTER

# SEVEN

# CONCLUSIONS

Give a concise summary of your research and finding here, and include a short summary of any future work as well.
Looking at this thing here, [2] [3] [4] it is a citation.

# REFERENCES


[1] *Musikalisches Würfelspiel*. In: *Wikipedia*. Sept. 27, 2025. URL: https://en.wikipedia.org/w/index.php?title=Musikalisches_W%C3%BCrfelspiel&oldid=1313693497 (visited on 03/24/2026).

[2] Roland van der Linden, Ricardo Lopes, and Rafael Bidarra. "Procedural Generation of Dungeons". In: *IEEE Transactions on Computational Intelligence and AI in Games* 6.1 (Mar. 2014), pp. 78–89. ISSN: 1943-0698. DOI: 10.1109/TCIAIG.2013.2290371. URL: https://ieeexplore.ieee.org/abstract/document/6661386 (visited on 02/26/2026).

[3] E. Galin et al. "Procedural Generation of Roads". In: *Computer Graphics Forum* 29.2 (2010), pp. 429–438. ISSN: 1467-8659. DOI: 10.1111/j.1467-8659.2009.01612.x. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1467-8659.2009.01612.x (visited on 02/26/2026).

[4] Johan Gangsås Hole. "Automatic Species Counterpoint". In: *Master Thesis at NTNU* (2026).

# APPENDICES

# A - GITHUB REPOSITORY

All code and latex-files used in this document are included in the Github repository linked below. Further explanations are given in the readme-file.

## Github repository link

- `https://github.com/lima98/TTK4900`

# B - AUDIO EXAMPLES

Audio-files for the examples in the thesis can be played from my NTNU-webpage.

## Audio example archive

- `https://www.janoivil.folk.ntnu.no`

# C - CODE DOCUMENTATION

All documentation for the code is included in the Github repository linked in 7 it is also provided as a web-page hosted at my NTNU-webpage.

## Code documentation

- `https://janoivil.folk.ntnu.no/docs/_build/html/index.html`